UNIT-2 (LINEAR DATA STRUCTURES)

ARRAY - 1D AND 2D

LECTURE NO:-7

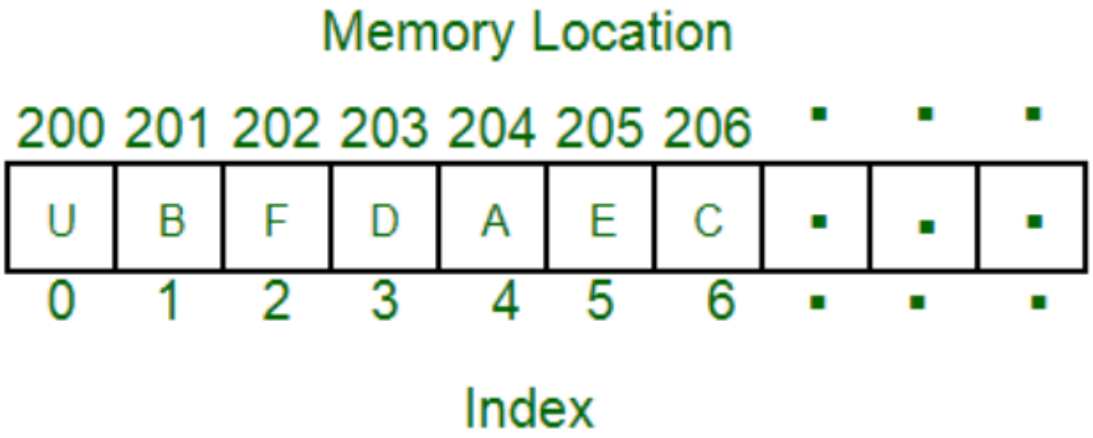
Array is a data structure that is used to store variables that are of similar data types at contiguous locations. The main advantage of the array is random access. There are mainly three types of the array:

- One Dimensional (1D) Array
- Two Dimension (2D) Array
- Multidimensional Array

One Dimensional Array:

1) It is a list of the variable of similar data types.
2) It allows random access and all the elements can be accessed with the help of their index.
3) The size of the array is fixed.

Representation of 1D array:



Two Dimensional Array:

1) It is a list of lists of the variable of the same data type.
2) It also allows random access and all the elements can be accessed with the help of their index.

Notes Prepared By: - Arvind Sharma (Assistant Professor, Department of CSE, MITRC Alwar)

3) It can also be seen as a collection of 1D arrays. It is also known as the Matrix.
4) Its dimension can be increased from 2 to 3 and 4 so on.
5) They all are referred to as a multi-dimension array.
6) The most common multidimensional array is a 2D array.

Representation of 2 D array:

	Column 0	Column 1	Column 2
Row 0	x[0][0]	x[0][1]	x[0][2]
Row 1	x[1][0]	x[1][1]	x[1][2]
Row 2	x[2][0]	x[2][1]	x[2][2]

Difference Table:

Basis	One Dimension Array	Two Dimension Array
Definition	Store a single list of the element of a similar data type.	Store a 'list of lists' of the element of a similar data type.
Representation	Represent multiple data items as a list.	Represent multiple data items as a table consisting of rows and columns.

Declaration	The declaration varies for different programming language: 1. For C++, ***datatype variable_name[row]*** 2. For Java, ***datatype [] variable_name= new datatype[row]***	The declaration varies for different programming language: 1. For C++, ***datatype variable_name[row] [column]*** 2. For Java, ***datatype [][] variable_name= new datatype[row][column]***
Dimension	One	Two
Size(bytes)	size of(datatype of the variable of the array) * size of the array	size of(datatype of the variable of the array)* the number of rows* the number of columns.
Address calculation.	Address of a[index] is equal to (base Address+ Size of each element of array * index).	Address of a[i][j] can be calculated in two ways row-major and column-major 1. **Column Major:** Base Address + Size of each element (number of rows(j-lower bound of the column)+ (i-lower bound of the rows)) 2. **Row Major:** Base Address + Size of each element (number of columns(i-lower bound of the row)+(j-lower bound of the column))

Example	int arr[5]; //an array with one row and five columns will be created.
	{a , b , c , d , e}
		int arr[2][5]; //an array with two rows and five columns will be created.
		a b c d e
		f g h i j

Applications of Arrays:

1) 2D Arrays are used to implement matrices.
2) Arrays can be used to implement various data structures like a heap, stack, queue, etc.
3) They allow random access.
4) They are cache-friendly.

MITRC

UNIT-2 (LINEAR DATA STRUCTURES)

STACK FUNDAMENTALS

LECTURE NO:- 8

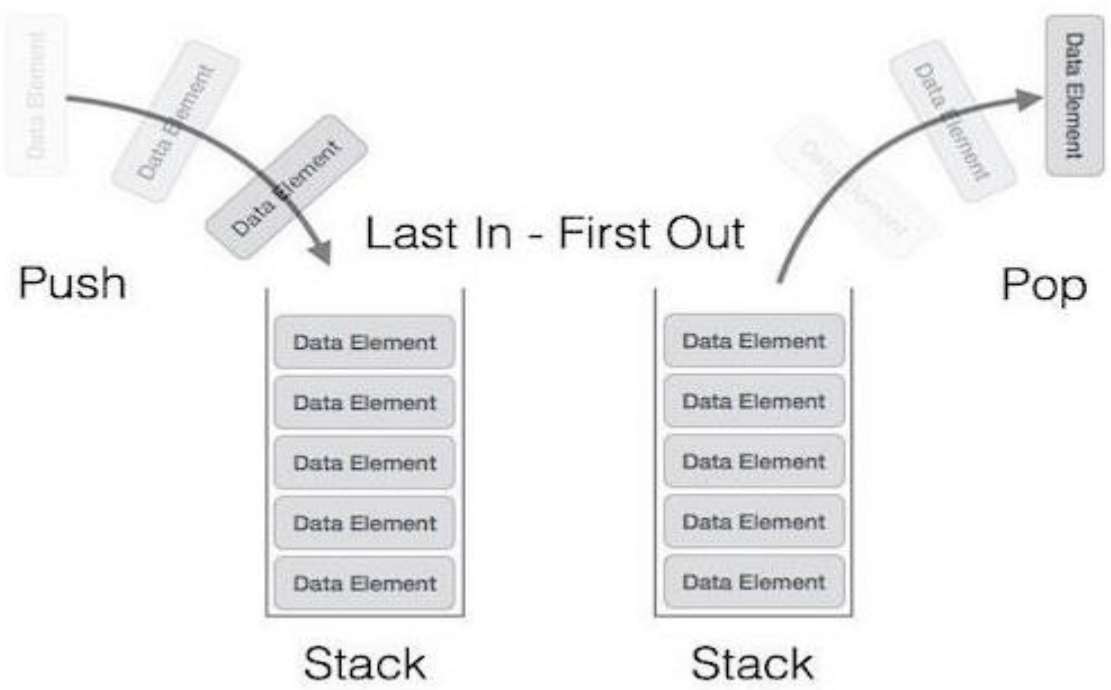
Stack

A stack is a linear data structure in which the insertion of a new element and removal of an existing element takes place at the same end represented as the top of the stack. To implement the stack, it is required to maintain the pointer to the top of the stack, which is the last element to be inserted because we can access the elements only on the top of the stack.

Stack Representation

A Stack ADT allows all data operations at one end only. At any given time, we can only access the top element of a stack.

The following diagram depicts a stack and its operations –

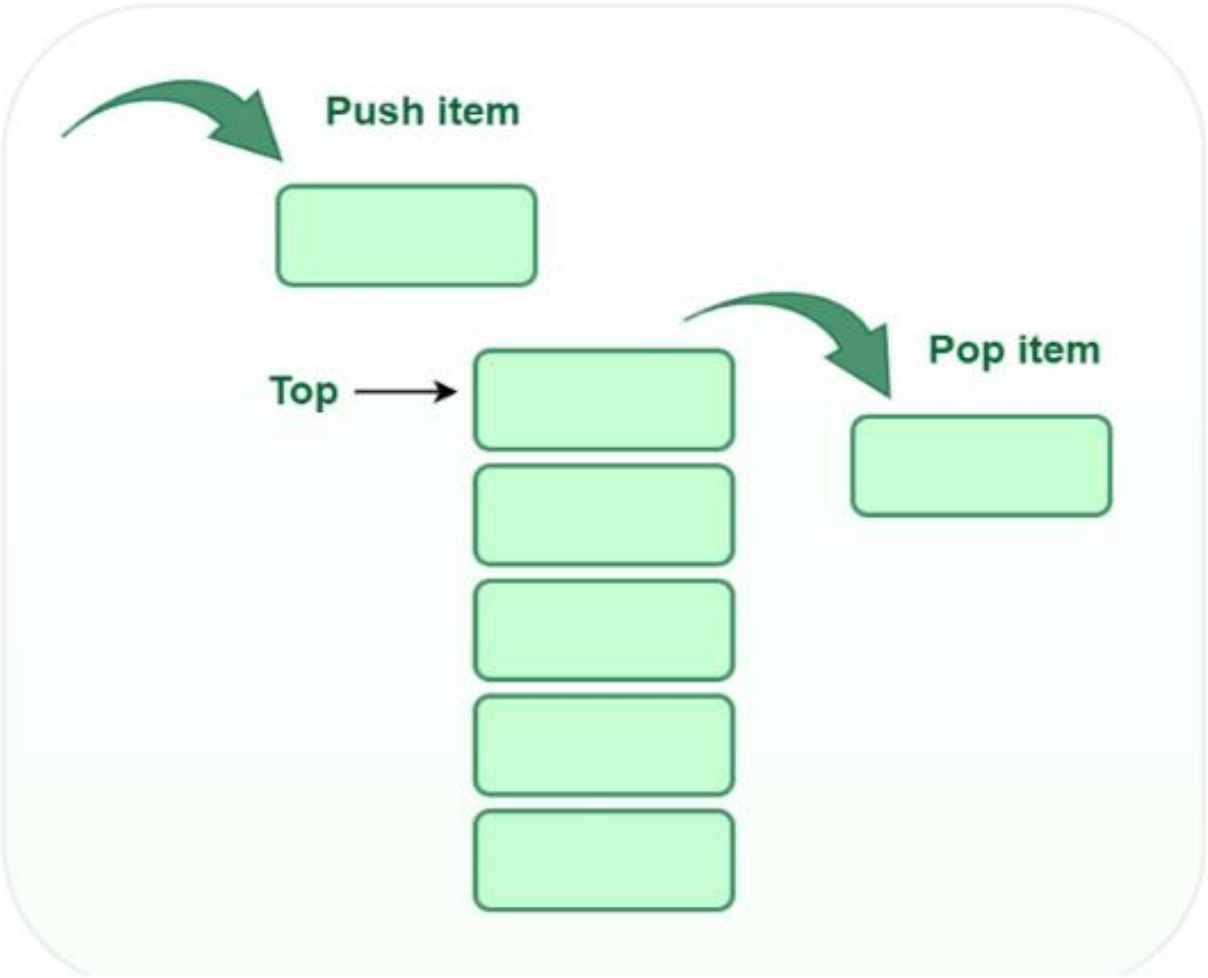


A stack can be implemented by means of Array, Structure, Pointer, and Linked List. Stack can either be a fixed size one or it may have a sense of dynamic resizing. Here, we are going to implement stack using arrays, which makes it a fixed size stack implementation.

Basic Operations on Stack

In order to make manipulations in a stack, there are certain operations provided to us.

- push() to insert an element into the stack
- pop() to remove an element from the stack
- top() Returns the top element of the stack.
- isEmpty() returns true if stack is empty else false.
- isFull() returns true if stack is full else false.
- size() returns the size of stack.



Push:

Adds an item to the stack. If the stack is full, then it is said to be an **Overflow condition.**

Algorithm for push:

```
begin
        if stack is full
                return
        endif
        else
                increment top
                stack[top] assign value
        end else
end procedure
```

Pop:

Removes an item from the stack. The items are popped in the reversed order in which they are pushed. If the stack is empty, then it is said to be an **Underflow condition.**

Algorithm for pop:

```
begin
        if stack is empty
                return
        endif
        else
                store value of stack[top]
                decrement top
                return value
        end else
end procedure
```

Top:

Returns the top element of the stack.

Algorithm for Top:

```
begin
        return stack[top]
end procedure
```

isEmpty:

Returns true if the stack is empty, else false.

Algorithm for isEmpty:

```
begin
        if top < 1
                return true
        else
                return false
end procedure
```

isFull:

Returns true if the stack is Full, else false.

Algorithm for isFull:

```
begin
        if top > size(no. of elements permissible) of array
                return true
        else
                return false
end procedure
```

UNIT-2 (LINEAR DATA STRUCTURES)

APPLICATIONS OF STACK : EXPRESSION EVALUATION

LECTURE NO:- 9

Applications of Stack

1. **Infix to Postfix**: Stack is used to convert infix expressions to postfix expressions.
2. **Expression Evaluation**: Stack is used to evaluate postfix expressions.
3. **Recursion**: Stack is used to keep track of function calls in a computer program and to return control to the correct function when a function returns.
4. **Memory Management**: Stack is used to store the values of the program counter and the values of the registers in a computer program, allowing the program to return to the previous state when a function returns.
5. **Balanced Parenthesis**: Stack is used to check the balance of parentheses in an expression.
6. **Undo-Redo**: Stack is used in computer programs to allow users to undo and redo actions.

Expression Evaluation

Evaluate an expression represented by a String. The expression can contain parentheses, we can assume parentheses are well-matched. For simplicity, you can assume only binary operations allowed are +, -, *, and /. Arithmetic Expressions can be written in one of three forms:

1. **Infix Notation:** Operators are written between the operands they operate on, e.g. 3 + 4.
2. **Prefix Notation:** Operators are written before the operands, e.g + 3 4
3. **Postfix Notation:** Operators are written after operands. e.g 3 4 +

UNIT-2 (LINEAR DATA STRUCTURES)

CONVERSION OF INFIX TO POSTFIX EXPRESSION

LECTURE NO:- 10

Infix expression: The expression of the form "a operator b" (a + b) i.e., when an operator is in-between every pair of operands.

Postfix expression: The expression of the form "a b operator" (ab+) i.e., When every pair of operands is followed by an operator.

Consider the following table showing infix to prefix and postfix conversion.

Infix Expression	Prefix Expression	Postfix Expression
A + B * C + D	+ + A * B C D	A B C * + D +
(A + B) * (C + D)	* + A B + C D	A B + C D + *
A * B + C * D	+ * A B * C D	A B * C D * +
A + B + C + D	+ + + A B C D	A B + C + D +

Examples:

Input: A + B * C + D

Output: ABC*+D+

Input: ((A + B) – C * (D / E)) + F

Output: AB+CDE/*-F+

1. Scan the infix expression **from left to right**.
2. If the scanned character is an operand, put it in the postfix expression.
3. Otherwise, do the following
    - If the precedence and associativity of the scanned operator are greater than the precedence and associativity of the operator in the stack [or the stack is empty or

Notes Prepared By: - Arvind Sharma (Assistant Professor, Department of CSE, MITRC Alwar)

the stack contains a '**(**' , then push it in the stack. ['**^**' operator is right associative and other operators like '**+**','**–**','*****' and '**/**' are left-associative].

- Check especially for a condition when the operator at the top of the stack and the scanned operator both are '**^**'. In this condition, the precedence of the scanned operator is higher due to its right associativity. So it will be pushed into the operator stack.
- In all the other cases when the top of the operator stack is the same as the scanned operator, then pop the operator from the stack because of left associativity due to which the scanned operator has less precedence.
- Else, Pop all the operators from the stack which are greater than or equal to in precedence than that of the scanned operator.
- After doing that Push the scanned operator to the stack. (If you encounter parenthesis while popping then stop there and push the scanned operator in the stack.)

4. If the scanned character is a '**(**', push it to the stack.
5. If the scanned character is a '**)**', pop the stack and output it until a '**(**' is encountered, and discard both the parenthesis.
6. Repeat steps **2-5** until the infix expression is scanned.
7. Once the scanning is over, Pop the stack and add the operators in the postfix expression until it is not empty.
8. Finally, print the postfix expression.

Example

Infix expression: K + L - M*N + (O^P) * W/U/V * T + Q

Input Expression	Stack	Postfix Expression
K		K
+	+	
L	+	K L

-	-	K L+
M	-	K L+ M
*	- *	K L+ M
N	- *	K L + M N
+	+	K L + M N* K L + M N* -
(	+ (	K L + M N *-
O	+ (	K L + M N * - O
^	+ (^	K L + M N* - O
P	+ (^	K L + M N* - O P
)	+	K L + M N* - O P ^
*	+ *	K L + M N* - O P ^
W	+ *	K L + M N* - O P ^ W
/	+ /	K L + M N* - O P ^ W *
U	+ /	K L + M N* - O P ^W*U
/	+ /	K L + M N* - O P ^W*U/
V	+ /	KL + MN*-OP^W*U/V
*	+ *	KL+MN*-OP^W*U/V/
T	+ *	KL+MN*-OP^W*U/V/T
+	+	KL+MN*-OP^W*U/V/T* KL+MN*-OP^W*U/V/T*+
Q	+	KL+MN*-OP^W*U/V/T*Q
		KL+MN*-OP^W*U/V/T*+Q+

UNIT-2 (LINEAR DATA STRUCTURES)

CONVERSION OF INFIX TO PREFIX EXPRESSION

LECTURE NO:- 11

Infix Expression: The expression of type **a 'operator' b** (a+b, where + is an operator) i.e., when the operator is between two operands.

Prefix Expression: The expression of type '**operator' a b** (+ab where + is an operator) i.e., when the operator is placed before the operands.

Examples

Input: A * B + C / D

Output: + * A B/ C D

Input: (A – B/C) * (A/K-L)

Output: *-A/BC-/AKL

1 Reverse the infix expression. Note while reversing each '(' will become ')' and each ')' becomes '('.

2 Scan the infix expression **from left to right**.

3 If the scanned character is an operand, put it in the postfix expression.

4 Otherwise, do the following

a. If the precedence and associativity of the scanned operator are greater than or equal to the precedence and associativity of the operator in the stack [or the stack is empty or the stack contains a '**)**' , then push it in the stack. ['**^**' operator is right associative and other operators like '**+**','**–**','*****' and '**/**' are left-associative].

i. Check especially for a condition when the operator at the top of the stack and the scanned operator both are '**^**'. In this condition, the precedence of the scanned operator is higher due to its right associativity. So it will be pushed into the operator stack.

Notes Prepared By: - Arvind Sharma (Assistant Professor, Department of CSE, MITRC Alwar)

MITRC

ii. In all the other cases when the top of the operator stack is the same as the scanned operator, then push the scanned operator to the stack

b. Else, Pop all the operators from the stack which are greater than to in precedence than that of the scanned operator.

i. After doing that Push the scanned operator to the stack. (If you encounter parenthesis while popping then stop there and push the scanned operator in the stack.)

4 If the scanned character is a '**)**', push it to the stack.

5 If the scanned character is a '**(**', pop the stack and output it until a '**(**' is encountered, and discard both the parenthesis.

6 Repeat steps **2-5** until the infix expression is scanned.

7 Once the scanning is over, Pop the stack and add the operators in the postfix expression until it is not empty.

8 Reverse the final postfix expression.

9 Finally, print the postfix expression.

Example

K + L - M * N + (O^P) * W/U/V * T + Q

If we are converting the expression from infix to prefix, we need first to reverse the expression.

The Reverse expression would be:

Q + T * V/U/W *) P^O(+ N*M - L + K

To obtain the prefix expression, we have created a table that consists of three columns, i.e., input expression, stack, and prefix expression. When we encounter any symbol, we simply add it into the prefix expression. If we encounter the operator, we will push it into the stack.

Input expression	Stack	Prefix expression
Q		Q
+	+	Q
T	+	QT

*	+*	QT
V	+*	QTV
/	+*/	QTV
U	+*/	QTVU
/	+*//	QTVU
W	+*//	QTVUW
*	+*//*	QTVUW
)	+*//*)	QTVUW
P	+*//*)	QTVUWP
^	+*//*)^	QTVUWP
O	+*//*)^	QTVUWPO
(	+*//*	QTVUWPO^
+	++	QTVUWPO^*//*
N	++	QTVUWPO^*//*N
*	++*	QTVUWPO^*//*N
M	++*	QTVUWPO^*//*NM
-	++-	QTVUWPO^*//*NM*
L	++-	QTVUWPO^*//*NM*L
+	++-+	QTVUWPO^*//*NM*L
K	++-+	QTVUWPO^*//*NM*LK
		QTVUWPO^*//*NM*LK+-++

UNIT-2 (LINEAR DATA STRUCTURES)

TOWER OF HANOI

LECTURE NO:- 12

Tower of Hanoi is a mathematical puzzle where we have three rods (**A**, **B**, and **C**) and **N** disks. Initially, all the disks are stacked in decreasing value of diameter i.e., the smallest disk is placed on the top and they are on rod **A**. The objective of the puzzle is to move the entire stack to another rod (here considered **C**), obeying the following simple rules:

- Only one disk can be moved at a time.
- Each move consists of taking the upper disk from one of the stacks and placing it on top of another stack i.e. a disk can only be moved if it is the uppermost disk on a stack.
- No disk may be placed on top of a smaller disk.

Examples

Input: 2

Output: Disk 1 moved from A to B

Disk 2 moved from A to C

Disk 1 moved from B to C

Input: 3

Output: Disk 1 moved from A to C

Disk 2 moved from A to B

Disk 1 moved from C to B

Disk 3 moved from A to C

Disk 1 moved from B to A

Disk 2 moved from B to C

Disk 1 moved from A to C

Notes Prepared By: - Arvind Sharma (Assistant Professor, Department of CSE, MITRC Alwar)

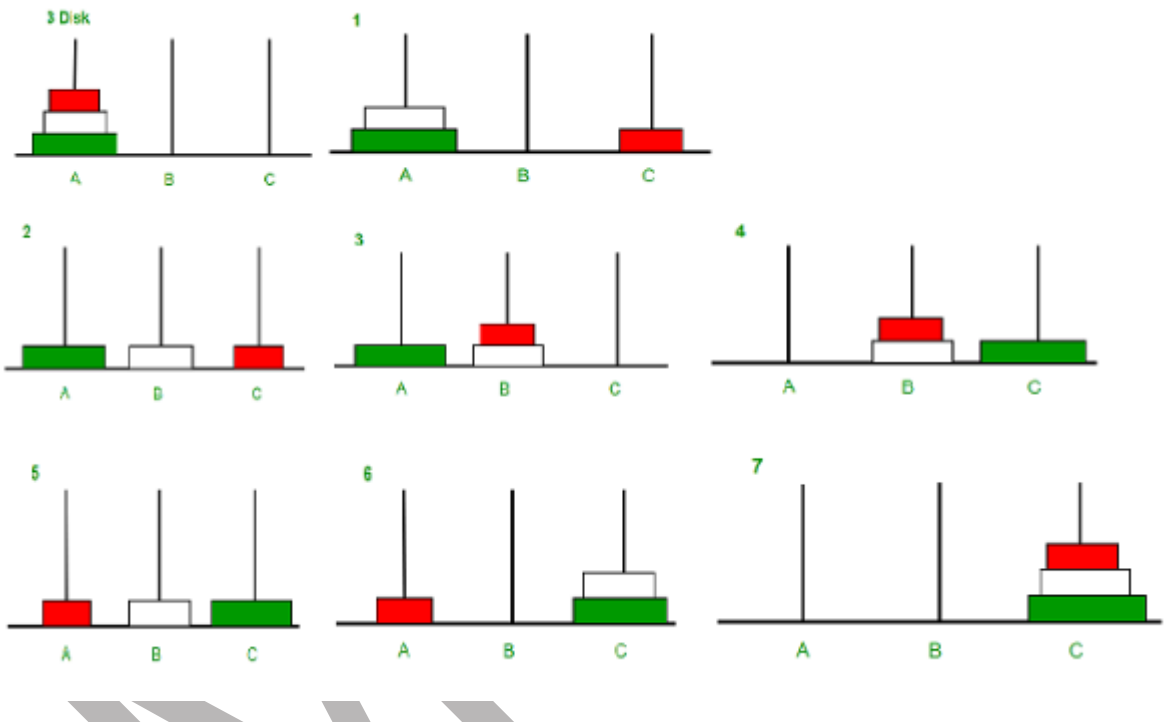
Tower of Hanoi using Recursion

The idea is to use the helper node to reach the destination using recursion. Below is the pattern for this problem:

Shift 'N-1' disks from 'A' to 'B', using C.

Shift last disk from 'A' to 'C'.

Shift 'N-1' disks from 'B' to 'C', using A.



Begin

```
towerOfHanoi(n, from_rod,  to_rod, aux_rod)
{
   if (n == 0) {
      return;
}
   towerOfHanoi(n - 1, from_rod, aux_rod, to_rod)
   print "Move disk " , n , from_rod , to_rod
   towerOfHanoi(n - 1, aux_rod, to_rod, from_rod)
}
```

End

UNIT-2 (LINEAR DATA STRUCTURES)

QUEUE - TYPES OF QUEUES: CIRCULAR QUEUE, DOUBLE ENDED QUEUE (DEQUEUE)

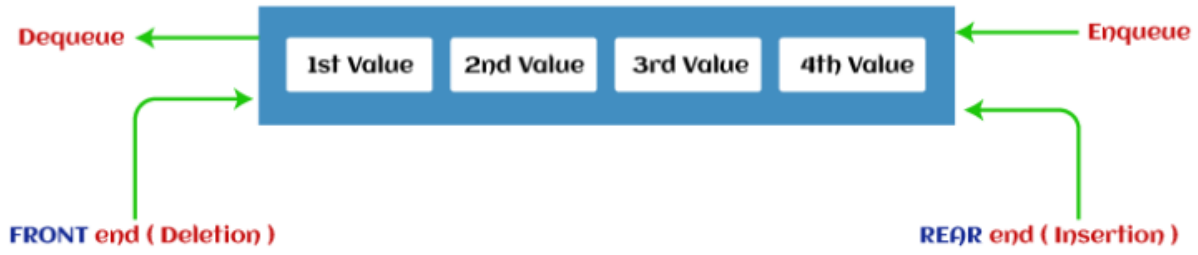
LECTURE NO:- 13

Queue

Queue is the data structure that is similar to the queue in the real world. A queue is a data structure in which whatever comes first will go out first, and it follows the FIFO (First-In-First-Out) policy. Queue can also be defined as the list or collection in which the insertion is done from one end known as the **rear end** or the **tail** of the queue, whereas the deletion is done from another end known as the **front end** or the **head** of the queue.

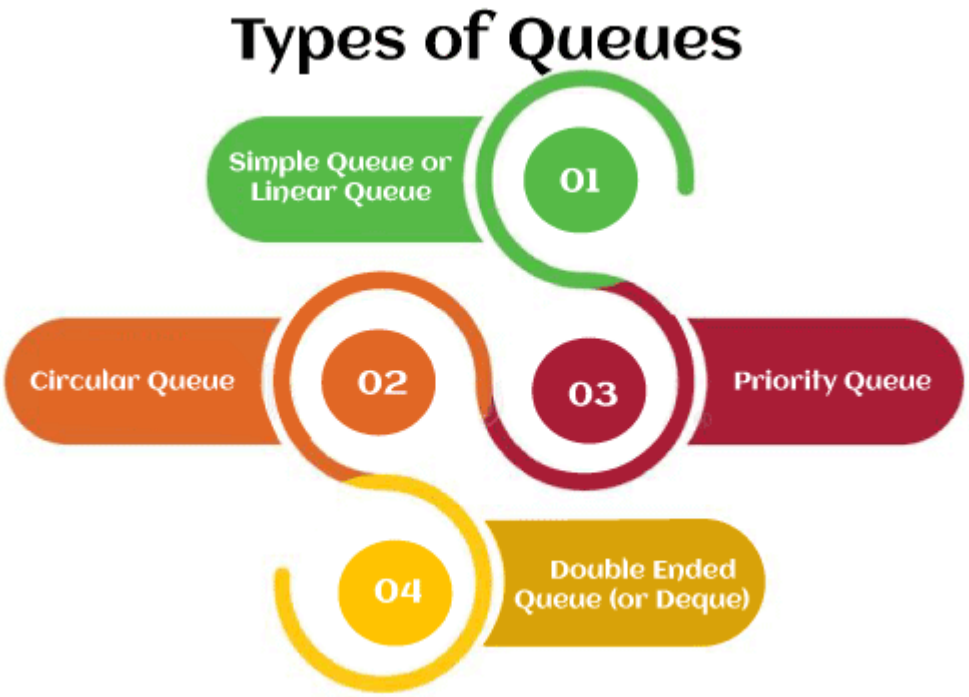
The real-world example of a queue is the ticket queue outside a cinema hall, where the person who enters first in the queue gets the ticket first, and the last person enters in the queue gets the ticket at last. Similar approach is followed in the queue in data structure.

The representation of the queue is shown in the below image –



Types of Queue

There are four different types of queue that are listed as follows -



Simple Queue or Linear Queue

In Linear Queue, an insertion takes place from one end while the deletion occurs from another end. The end at which the insertion takes place is known as the rear end, and the end at which the deletion takes place is known as front end. It strictly follows the FIFO rule.

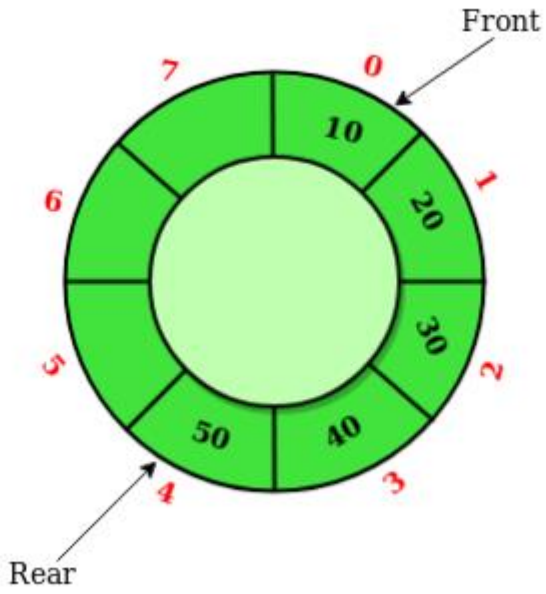


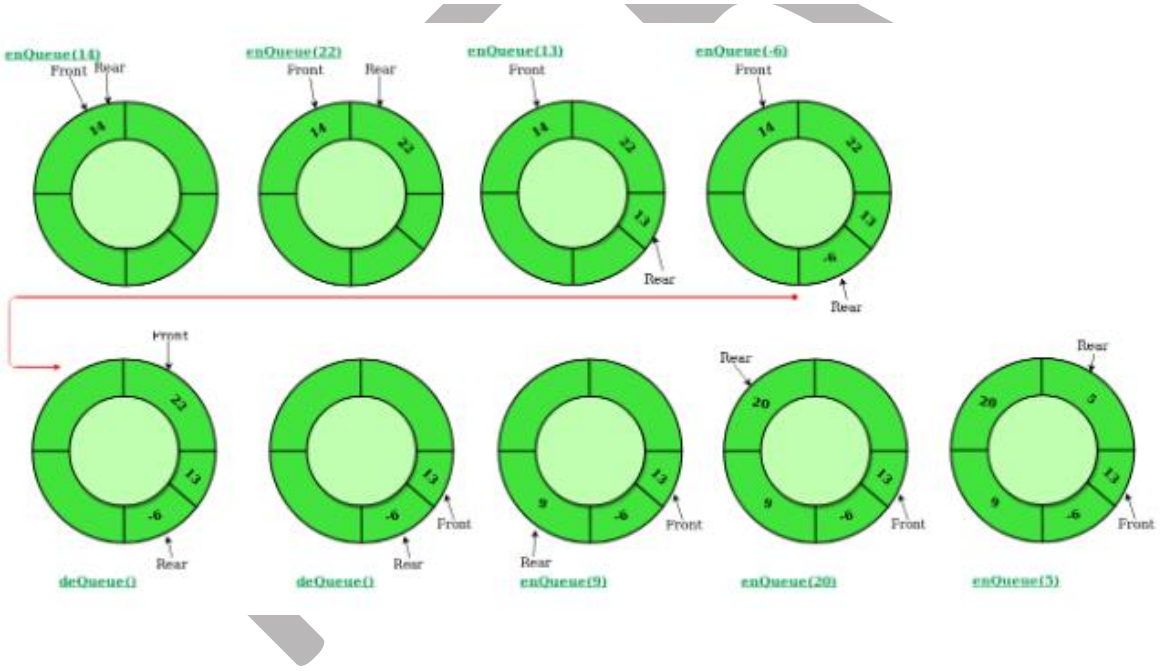
The major drawback of using a linear Queue is that insertion is done only from the rear end. If the first three elements are deleted from the Queue, we cannot insert more elements even though the space is available in a Linear Queue. In this case, the linear Queue shows the overflow condition as the rear is pointing to the last element of the Queue.

Circular Queue

A Circular Queue is an extended version of a normal queue where the last element of the queue is connected to the first element of the queue forming a circle.

The operations are performed based on FIFO (First In First Out) principle. It is also called **'Ring Buffer'**.

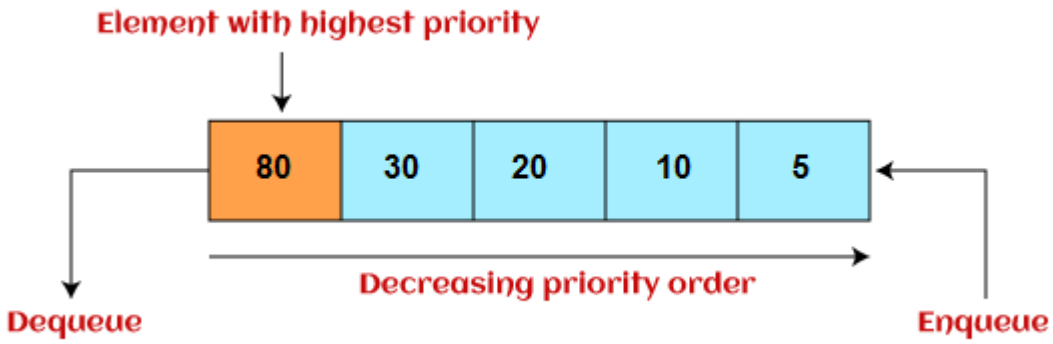




The drawback that occurs in a linear queue is overcome by using the circular queue. If the empty space is available in a circular queue, the new element can be added in an empty space by simply incrementing the value of rear. The main advantage of using the circular queue is better memory utilization.

Priority Queue

It is a special type of queue in which the elements are arranged based on the priority. It is a special type of queue data structure in which every element has a priority associated with it. Suppose some elements occur with the same priority, they will be arranged according to the FIFO principle. The representation of priority queue is shown in the below image -



Insertion in priority queue takes place based on the arrival, while deletion in the priority queue occurs based on the priority. Priority queue is mainly used to implement the CPU scheduling algorithms.

There are two types of priority queue that are discussed as follows -

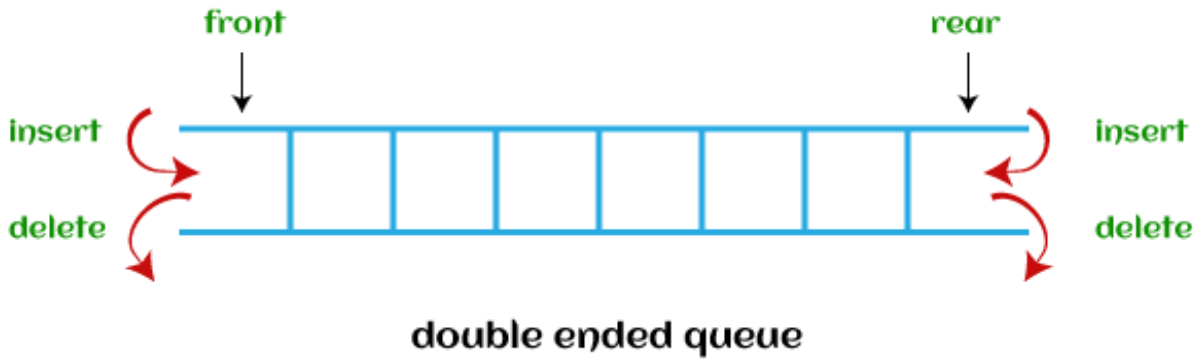
- **Ascending priority queue -** In ascending priority queue, elements can be inserted in arbitrary order, but only smallest can be deleted first. Suppose an array with elements 7, 5, and 3 in the same order, so, insertion can be done with the same sequence, but the order of deleting the elements is 3, 5, 7.
- **Descending priority queue -** In descending priority queue, elements can be inserted in arbitrary order, but only the largest element can be deleted first. Suppose an array with elements 7, 3, and 5 in the same order, so, insertion can be done with the same sequence, but the order of deleting the elements is 7, 5, 3.

Deque (Double Ended Queue)

In Deque or Double Ended Queue, insertion and deletion can be done from both ends of the queue either from the front or rear. It means that we can insert and delete elements from both front and rear ends of the queue. Deque can be used as a palindrome checker means that if we read the string from both ends, then the string would be the same.

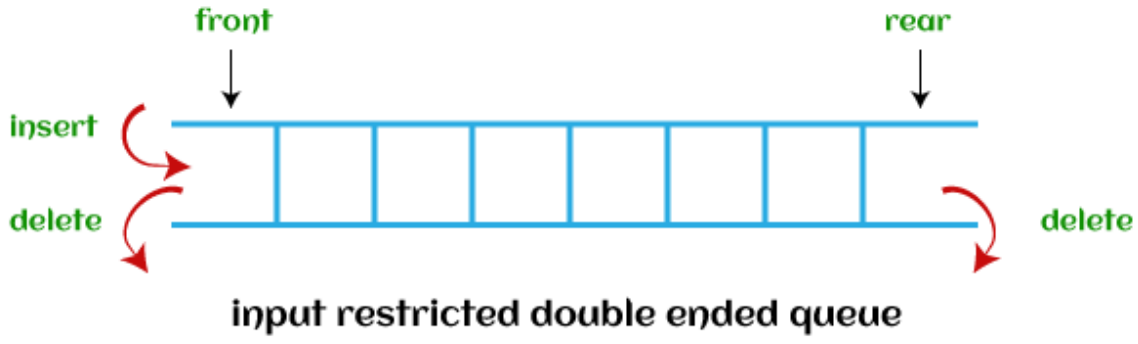
Deque can be used both as stack and queue as it allows the insertion and deletion operations on both ends. Deque can be considered as stack because stack follows the LIFO (Last In First Out) principle in which insertion and deletion both can be performed only from one end. And in deque, it is possible to perform both insertion and deletion from one end, and Deque does not follow the FIFO principle.

The representation of the deque is shown in the below image -

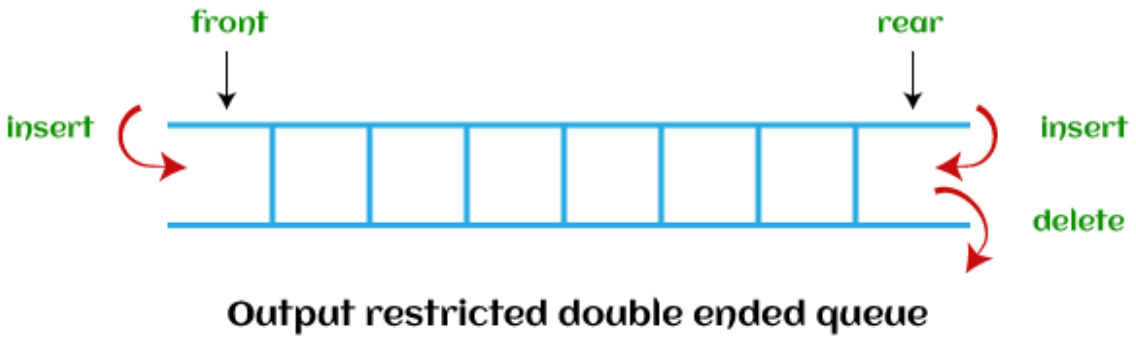


There are two types of deque that are discussed as follows -

Input restricted deque - As the name implies, in input restricted queue, insertion operation can be performed at only one end, while deletion can be performed from both ends.



Output restricted deque - As the name implies, in output restricted queue, deletion operation can be performed at only one end, while insertion can be performed from both ends.



Output restricted double ended queue

Operations performed on queue

The fundamental operations that can be performed on queue are listed as follows -

- **Enqueue:** The Enqueue operation is used to insert the element at the rear end of the queue.
- **Dequeue:** It performs the deletion from the front-end of the queue. It also returns the element which has been removed from the front-end.
- **Peek:** This is the third operation that returns the element, which is pointed by the front pointer in the queue but does not delete it.
- **Queue overflow (isFull):** It shows the overflow condition when the queue is completely full.
- **Queue underflow (isEmpty):** It shows the underflow condition when the Queue is empty, i.e., no elements are in the Queue.

Implementation the Queue

There are two ways of implementing the Queue:

- **Implementation using array:** The sequential allocation in a Queue can be implemented using an array.
- **Implementation using Linked list:** The linked list allocation in a Queue can be implemented using a linked list.

UNIT-2 (LINEAR DATA STRUCTURES)

APPLICATIONS – PRIORITY QUEUE USING ARRAYS

LECTURE NO:- 14

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 10

void create_queue();
void insert_element(int);
void delete_element(int);
void check_priority(int);
void display_priorityqueue();

int pqueue[MAX];
int front, rear;

void main()
{
   int n, choice;
   printf("\nEnter 1 to insert element by priority ");
   printf("\nEnter 2 to delete element by priority ");
   printf("\nEnter 3 to display priority queue ");
   printf("\nEnter 4 to exit");
   create_queue();
   while (1)
   {
      printf("\nEnter your choice : ");
```

MITRC

```
      scanf("%d", &choice);
      switch(choice)
      {
      case 1:
         printf("\nEnter element to insert : ");
         scanf("%d",&n);
         insert_element(n);
         break;
      case 2:
         printf("\nEnter element to delete : ");
         scanf("%d",&n);
         delete_element(n);
         break;
      case 3:
         display_priorityqueue();
         break;
      case 4:
         exit(0);
      default:
         printf("\n Please enter valid choice");
      }
   }
}
void create_queue()
{
   front = rear = -1;
}
void insert_element(int data)
{
```

MITRC

```
    if (rear >= MAX - 1)
    {
        printf("\nQUEUE OVERFLOW");
        return;
    }
    if ((front == -1) && (rear == -1))
    {
        front++;
        rear++;
        pqueue[rear] = data;
        return;
    }
    else
        check_priority(data);
    rear++;
}
void check_priority(int data)
{
    int i,j;
    for (i = 0; i <= rear; i++)
    {
        if (data >= pqueue[i])
        {
            for (j = rear + 1; j > i; j--)
            {
                pqueue[j] = pqueue[j - 1];
            }
            pqueue[i] = data;
            return;
```

MITRC

```
      }
   }
   pqueue[i] = data;
}
void delete_element(int data)
{
   int i;
   if ((front==-1) && (rear==-1))
   {
      printf("\nEmpty Queue");
      return;
   }
   for (i = 0; i <= rear; i++)
   {
      if (data == pqueue[i])
      {
         for (; i < rear; i++)
         {
           pqueue[i] = pqueue[i + 1];
         }
         pqueue[i] = -99;
         rear--;
         if (rear == -1)
           front = -1;
         return;
      }
   }
   printf("\n%d element not found in queue", data);
}
```

MITRC

```
void display_priorityqueue()
{
   if ((front == -1) && (rear == -1))
   {
      printf("\nEmpty Queue ");
      return;
   }
   for (; front <= rear; front++)
   {
      printf(" %d ", pqueue[front]);
   }
    front = 0;
}
```

MITRC

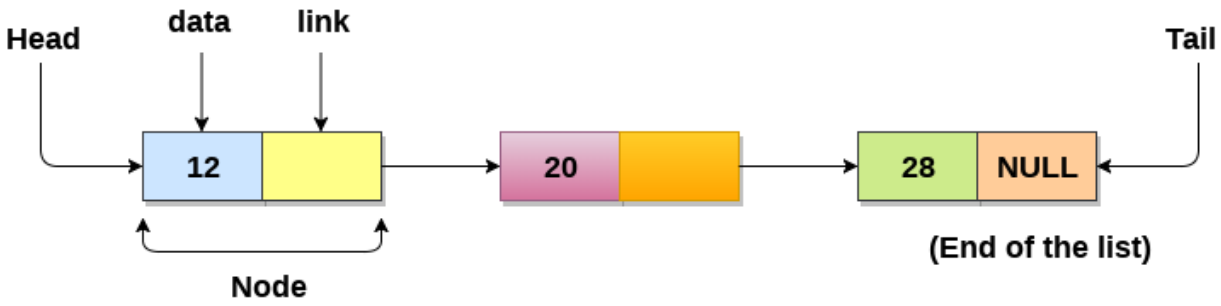
UNIT-2 (LINEAR DATA STRUCTURES)

LIST - SINGLY LINKED LISTS – DOUBLY LINKED LISTS - CIRCULAR LINKED LISTS

LECTURE NO:- 15

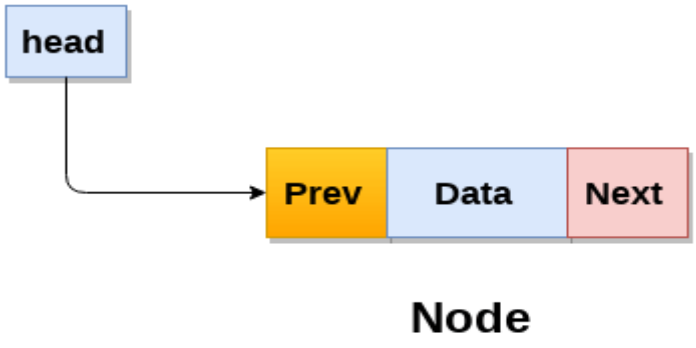
Singly Linked List

Linked List can be defined as collection of objects called **nodes** that are randomly stored in the memory. A node contains two fields i.e. data stored at that particular address and the pointer which contains the address of the next node in the memory. The last node of the list contains pointer to the null.

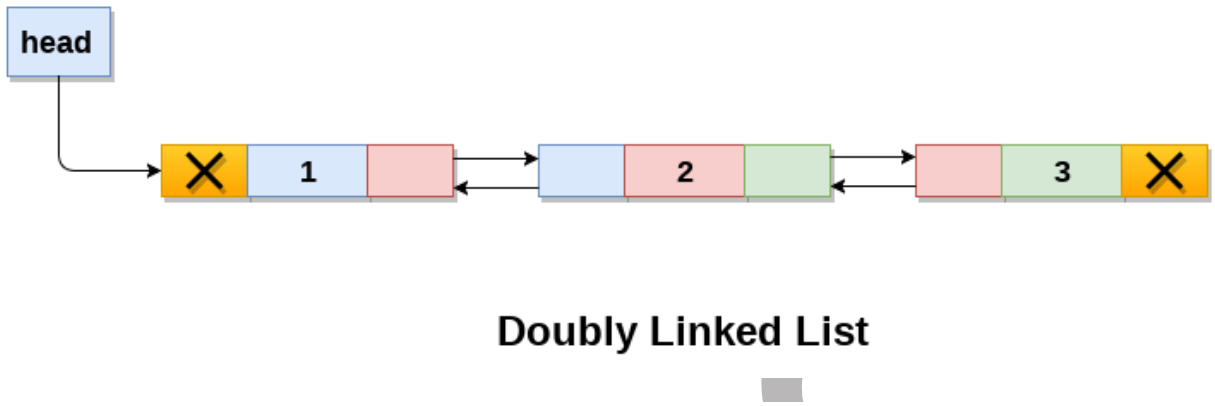


Doubly Linked List

Doubly linked list is a complex type of linked list in which a node contains a pointer to the previous as well as the next node in the sequence. Therefore, in a doubly linked list, a node consists of three parts: node data, pointer to the next node in sequence (next pointer) , pointer to the previous node (previous pointer). A sample node in a doubly linked list is shown in the figure.



A doubly linked list containing three nodes having numbers from 1 to 3 in their data part, is shown in the following image.

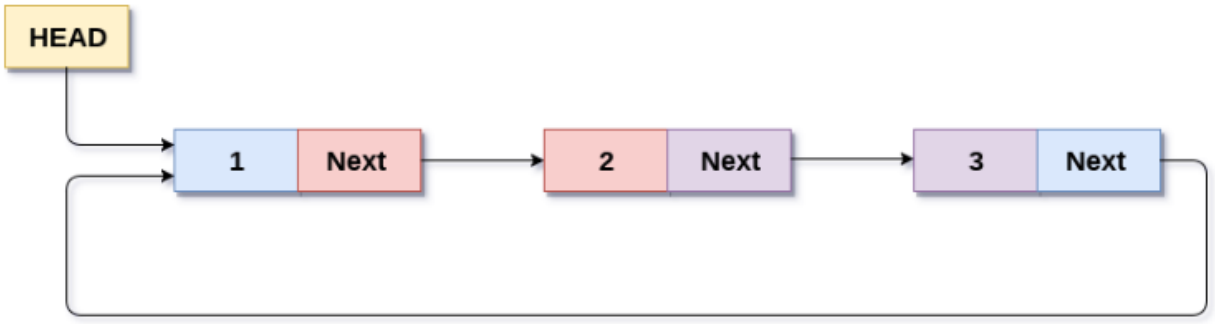


Doubly Linked List

Circular Singly Linked List

In a circular Singly linked list, the last node of the list contains a pointer to the first node of the list. We can have circular singly linked list as well as circular doubly linked list. We traverse a circular singly linked list until we reach the same node where we started. The circular singly liked list has no beginning and no ending. There is no null value present in the next part of any of the nodes.

The following image shows a circular singly linked list.



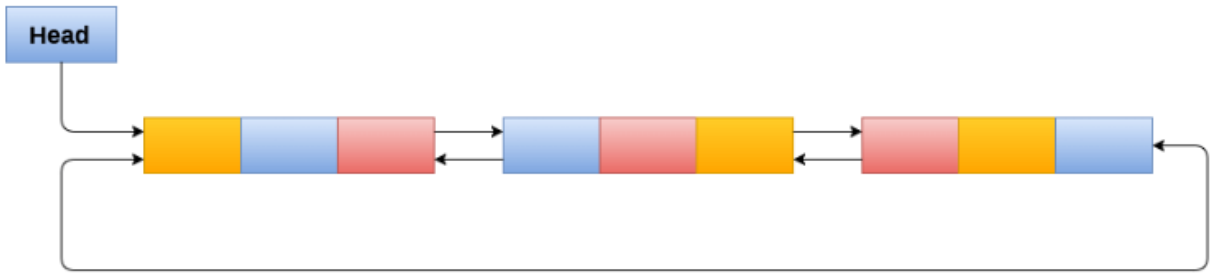
Circular Singly Linked List

Circular linked list are mostly used in task maintenance in operating systems. There are many examples where circular linked list are being used in computer science including browser surfing where a record of pages visited in the past by the user, is maintained in the form of circular linked lists and can be accessed again on clicking the previous button.

Circular Doubly Linked List

Circular doubly linked list is a more complexed type of data structure in which a node contain pointers to its previous node as well as the next node. Circular doubly linked list doesn't contain NULL in any of the node. The last node of the list contains the address of the first node of the list. The first node of the list also contain address of the last node in its previous pointer.

A circular doubly linked list is shown in the following figure.



Circular Doubly Linked List

Due to the fact that a circular doubly linked list contains three parts in its structure therefore, it demands more space per node and more expensive basic operations. However, a circular doubly linked list provides easy manipulation of the pointers and the searching becomes twice as efficient.

There are following Major Operations on Linked List

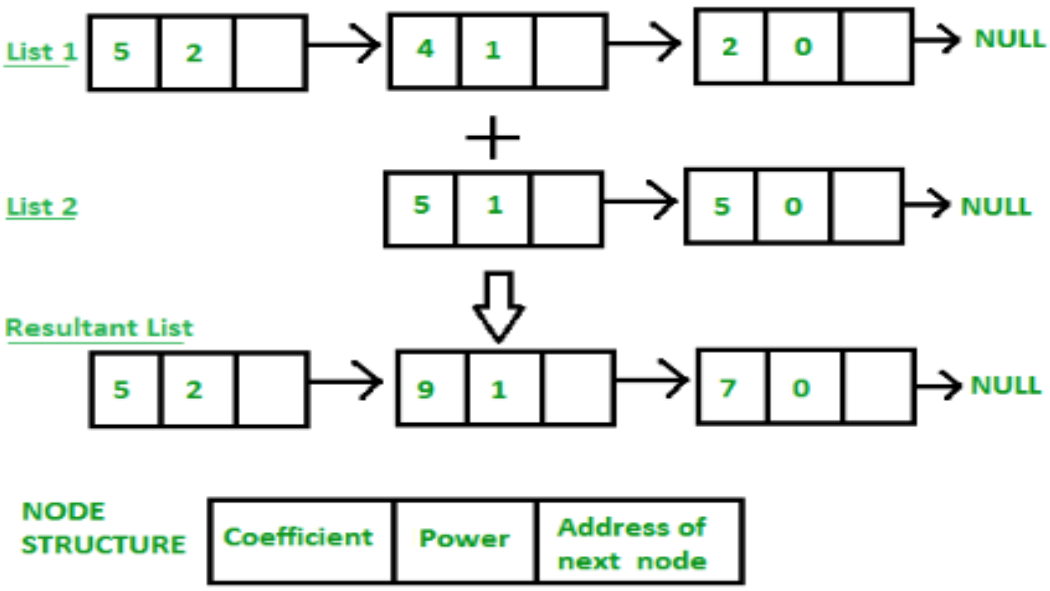
1) Insertion
2) Deletion
3) Searching
4) Traversing

UNIT-2 (LINEAR DATA STRUCTURES)

APPLICATIONS OF LIST -- POLYNOMIAL ADDITION / SUBTRACTION

LECTURE NO:- 16

We can perform the polynomial addition and subtraction using linked list.



Examples

Input:

1st number = $5x^2 + 4x^1 + 2x^0$

2nd number = $-5x^1 - 5x^0$

Output:

$5x^2-1x^1-3x^0$

Input:

1st number = $5x^3 + 4x^2 + 2x^0$

2nd number = 5x^1 - 5x^0

Output:

$5x^3 + 4x^2 + 5x^1 - 3x^0$